Python 3.12 AsyncIO & High-Performance Coroutines • STUDENT EXAM COPY

Course: Python 3.12.8 • College: DPU • Student Name: _____

Q1. In Python 3.12, which flag enforces immediate failure whenever a DeprecationWarning is triggered?

A) -W error::DeprecationWarning

B) --ignore-deprecation

C) -O2

D) --strict-typing

Q2. What is the primary function of Python's asyncio.gather(*tasks)?

A) Concurrently await multiple coroutines and return their aggregated results

B) Spawn OS child processes across multi-core processors

C) Compile Python functions to native assembly code

D) Encrypt coroutine payloads using TLS

Q3. Why is deterministic output normalization critical in automated code verification for Python?

A) It masks memory addresses, timestamps, and thread IDs so identical logic yields identical results

B) It rewrites slow algorithms into faster ones automatically

C) It disables unit testing to make test suites run faster

D) It converts console output into binary machine code

Q4. What is the primary objective of mutation testing in exercise validation?

A) Proving that student-facing test suites successfully fail when code logic is intentionally mutated/broken

B) Randomly deleting student files to test backup recovery

C) Speeding up Docker container build times

D) Encrypting source code with AES-256

Q5. In Python, what is the main consequence of executing untrusted code without container sandbox isolation?

A) Direct host filesystem compromise, resource exhaustion, and external network leaks

B) Syntax highlighting colors changing randomly

C) The compiler running 100x faster than expected

D) PowerPoint slide fonts changing to Comic Sans

Q6. When configuring container limits for isolated verification, why is '--network none' strictly enforced?

A) To prevent code from downloading malicious payloads, leaking credentials, or contacting external servers

B) Because Docker containers cannot support networking on Linux

C) To decrease the size of Docker image layers on disk

D) To disable JSON serialization in the API

Q7. What is the recommended approach for handling deprecation warnings in current production Python?

A) Treat warnings as errors in CI/CD pipelines and migrate to the approved modern replacements

B) Pin older runtime versions indefinitely to avoid updating

C) Turn off compiler error logging across the entire company

D) Use reflection to bypass deprecated method signatures

Q8. Which practice is recommended when engineering scalable Python applications for Python 3.12 AsyncIO & High-Performance Coroutines?

A) Use verified idioms, explicit types, and automated unit test assertions

B) Suppress all compiler and deprecation warnings to simplify logs

C) Rely entirely on global mutable state to avoid passing parameters

D) Execute all database calls synchronously inside UI or main thread loops

Q9. What is the exact output produced by executing this Python code?

```
nums = [1, 2, 3, 4]
res = [x * 2 for x in nums if x % 2 == 0]
print(res)
```

A) Compilation Error / Syntax Warning

B) [4, 8]

C) null

D) Runtime Exception: IndexOutOfBounds

Q10. What is the exact output produced by executing this Python code?

```
data = {'a': 10, 'b': 20}
val = data.get('c', 30) + data['a']
print(val)
```

A) Compilation Error / Syntax Warning

B) 40

C) null

D) Runtime Exception: IndexOutOfBounds